

Modül 01: Başlarken

Kod yazmayı zaten biliyorsunuz. Bu modül makinenizde Java'yı çalıştırır, bir iş yapan en küçük programı gösterir ve onu çalıştırdığınızda gerçekte ne olduğunu anlatır. Yarım saat sürer ve JDK, JRE ile JVM arasındaki farkı bir daha asla karıştırmazsınız.

Neler Öğreneceksiniz

- JDK 27'yi nasıl çalışır hale getireceğiniz ve doğru sürümde olduğunuzu nasıl doğrulayacağınız
- JVM, JRE ve JDK'nın her birinin ne olduğu ve insanların neden karıştırdığı
- Sınıf bildirimi ya da build aracı olmadan Java programı yazıp çalıştırma
- Derleyicinin dosyanızla ne yaptığı, iddia edilerek değil gösterilerek
- Bir derleyici hatasını okuyup şikâyet ettiği tam noktayı bulma

Başka Bir Dilden Geliyorsanız

Python ya da JavaScript'te bir dosya yazıp çalıştırırsınız. Tek adım. Java geleneksel olarak size iki adım yaptırdı: kaynağı bytecode'a derlemek, sonra bytecode'u çalıştırmak. "Java hantaldır" ününün başladığı yer bu ek adımdır.

Bu iki şeyin ikisi de değişti ve bu modül değişimle tanıştığınız yer.

	Python / JavaScript	Java, geleneksel olarak	Java 27
Script çalıştırma	<code>python app.py</code>	<code>javac App.java</code> sonra <code>java App</code>	<code>java App.java</code>
En küçük program	<code>print("hi")</code>	<code>sınıf + static main + String[] args</code>	<code>void main() + IO.println</code>
Hataların bulunması	çalışma zamanında	derleme zamanında	derleme zamanında

Çalışma biçiminizi asıl değiştirecek olan son satır. Python, nadiren girilen bir dalda yazım hatası olduğunu size kullanıcı üretimde çarpınca söyler. Java program daha başlamadan söyler. Yaptığınız takas budur: baştan biraz daha tören, sonrasında çok daha az sürpriz.

Ders

Üç harfli kelimeler

İnsanlar JVM, JRE ve JDK'yı birbirinin yerine kullanıyor ve sonra birbirlerini karıştırıyorlar. Bunlar üst üste duran üç ayrı şey:

- **JVM**, Java Virtual Machine. Bytecode'u çalıştıran şey. Derlenmiş Java'nın macOS, Linux ve Windows'ta değişmeden çalışmasının sebebi. Kodunuz bir işlemciyi değil, bunu hedefler.
- **JRE**, Java Runtime Environment. JVM artı standart kütüphane. Java'yı *çalıştırmaya* yeter, derlemeye yetmez. Artık nadiren ayrı kurulur.
- **JDK**, Java Development Kit. JRE artı araçlar: derleyici `javac`, sökücü `javap`, `jar`, `javadoc`. Kurduğunuz şey budur.

Siz bir JDK kurdunuz. Doğrulayın:

```
java -version
```

`java version "27"` görmelisiniz. Başka bir şeyse buradaki örnekler dersle hiç ilgisi olmayan sebeplerle başarısız olabilir.

En küçük program

`examples/HelloWorld.java` dosyasını açın. Dosyanın tamamı:

```
void main() {  
    IO.println("Hello, World!");  
}
```

Daha önce Java gördüyseniz, orada olmayanları fark edin. `public class` yok, `static` yok, `String[] args` yok, yazdırma için `import` yok. Bu tören yirmi beş yıl boyunca Java'nın en çok alay edilen özelliği idi ve JEP 512, JDK 25'te onu yazma zorunluluğunu kaldırdı. Preview değil, kararlı.

`IO` `import` olmadan erişilebilir çünkü `java.lang` içinde yaşar ve her Java dosyası onu otomatik olarak içe aktarır.

Tören kaybolmadı

Burada yavaşlamaya değer, çünkü Modül 05'te sizi ısıracak bir yanlış anlamayı önler.

Java yeni başlayanlar için ikinci ve daha basit bir dil edinmedi. Töreni sizin yerinize derleyici yazıyor. Kanıtlayın:

```
javac -d /tmp/java27 modules/01-getting-started/examples/WhatTheCompilerWrites.java  
javap -cp /tmp/java27 WhatTheCompilerWrites
```

Dönen çıktı:

```
final class WhatTheCompilerWrites {
    WhatTheCompilerWrites();
    void main();
}
```

İşte sınıfınız. Derleyici onu dosyadan sonra adlandırdı, `final` işaretledi, argümansız bir kurucu verdi ve `main` 'i bir **örnek** metodu olarak bıraktı. Özelliğin adının "instance main methods" olmasının sebebi bu: çalışma zamanı bu sınıftan bir nesne kurar ve `main` 'i onun üzerinde çağırır.

Yani baştan sona sıradan Java yazıyorsunuz, sadece daha azını yazarak. Buradaki her şey Modül 05'teki klasik forma aktarılır, çünkü zaten o formun kendisi.

Çalıştırmak gerçekte ne yapar

`java HelloWorld.java` çalıştırdığınızda sırayla üç şey olur:

1. `javac` kaynağınızı JVM'in anladığı bir komut kümesi olan **bytecode**'a derler. Java 11'den beri bu bellekte olabiliyor, `.class` dosyası görmemenizin sebebi bu.
2. JVM o bytecode'u yükler ve doğrular.
3. JVM onu çalıştırır, sıcak yolları ilerledikçe yerel makine koduna derler.

Birinci adım yazım hatalarınızın yakalandığı yer. Derlenen, statik tipli bir dilin tüm pazarlığı budur: derleyici, hiçbir çalışmadan önce her satırı okur.

Uzun yoldan da yapabilirsiniz, ve bazen isteyeceksiniz:

```
javac Greet.java    # Greet.class üretir
java Greet          # dikkat: .java yok, .class yok, sadece sınıf adı
```

Derleyicinin size söylediğini okumak

Yeni bir dildeki ilk gerçek beceriniz kod yazmak değil. Hatayı okuyup nereye bakacağınızı bilmektir.

`examples/ReadTheError.java` bilerek bozuk. Çalıştırın:

```
ReadTheError.java:29: error: ';' expected
    IO.println("I am missing something")
                        ^
1 error
```

Şapka işaretinden geriye doğru okuyun:

- `^` tam sütunu gösterir
- `:29` satır
- `';' expected` derleyicinin orada bulmayı beklediği şey

Java niyetinizi tahmin etmiyor. Kodunuzu o şapkaya kadar ayrıştırdı ve o konumda tek geçerli belirteç noktalı virgüldü. Şapkanın gösterdiği yeri düzeltin, tüm satırı değil.

Çalıştırın

```
# En küçük program
java modules/01-getting-started/examples/HelloWorld.java

# Derleyicinin sizin için ürettiğini görün
java modules/01-getting-started/examples/WhatTheCompilerWrites.java

# Etkileşimli, bunu kendiniz çalıştırın
java modules/01-getting-started/examples/AskingForInput.java

# Bilerek bozuk. Hatayı okuyun.
java modules/01-getting-started/examples/ReadTheError.java

# Bu modüldeki her örneğin hâlâ doğru davrandığını kontrol edin
./scripts/verify-examples.sh modules/01-getting-started
```

Sık Yapılan Hatalar

Elinizde yalnızca `Greet.java` varken `java Greet` çalıştırmak.

```
Error: Could not find or load main class Greet
Caused by: java.lang.ClassNotFoundException: Greet
```

`java Greet`, `Greet` adında derlenmiş bytecode arar. `java Greet.java` kaynağı çalıştırır. Fark `.java` uzantısında, ve hata mesajı bunu size söylemiyor.

`I0.println` yerine `io.println` yazmak.

```
error: cannot find symbol
    io.println("hi");
    ^
symbol:   variable io
location: class E3
```

Java büyük küçük harfe duyarlıdır ve `I0` bir sınıf adıdır. "Cannot find symbol" ilk ayınızda en sık göreceğiniz hatadır. Neredeyse her zaman bir yazım hatası, eksik bir import ya da henüz var olmayan bir ad demektir.

Sol taraf metin olduğunda `+` işaretinin toplayacağını sanmak.

```
"Next year is " + year + 1    // "Next year is 2026"
"Next year is " + (year + 1)  // "Next year is 2027"
```

`+` soldan sağa değerlendirilir. Bir taraf `String` olduğu anda ondan sonraki her `+` "topla" değil "sona ekle" demektir. Aritmetiği parantez içine alın.

`System.out.println` artık geçersiz sanmak. Değil. `IO.println`, kompakt kaynak dosyalarında bulunan bir kısaltma. Okuyacağınız her gerçek kod tabanı `System.out.println` ya da bir loglama çatısı kullanıyor, ve ikisi de burada çalışıyor.

Özet Çıkarımlar

- **Kurduğunuz şey JDK'dır.** Bytecode çalıştıran JVM'i ve onu üreten araçları içerir. Bu müfredat için `java -version` çıktısı 27 demelidir.
- **`java File.java` bir kaynak dosyasını doğrudan çalıştırır.** Build aracı yok, ayrı derleme adımı yok, geride `.class` dosyası kalmaz.
- **Kompakt kaynak dosyaları gerçek Java'dır.** Sınıf bildirimini, kurucuyu ve `final` niteleyicisini derleyici sizin için yazar. `javap` size tam olarak ne yazdığını gösterir.
- **Buradaki `main` bir örnek metodudur,** dolayısıyla `this` vardır. Klasik `public static void main` bunu taşımaz, Modül 05 nedenini anlatıyor.
- **Derleme hataları belirsiz değil, konumludur.** Şapka sütunu, numara satırı verir, mesaj da derleyicinin ne beklediğini söyler.
- **Hiçbir şey değiştirilmedi.** `System.out.println`, sınıflar ve `static main` hepsi hâlâ çalışıyor. Farklı bir dil değil, daha az yazmayı öğreniyorsunuz.

Ödev

[homework/README.md](#)

Az önce gördüklerinizden küçük etkileşimli bir program kurun, sonra bilerek bozup derleyicinin ne dediğini okuyun. Referans çözüm [solutions/01-getting-started/](#) altında, ama önce kendiniz deneyin. Cevabı okumak, hatayı alıp düzeltmekten çok daha az şey öğretir.